

Week 3 Assignment — The Governed AI Pipeline

Anna Kaszyńska

2026-05-31

GitHub Repository: <https://github.com/kaszynskaa/spring-boot-realworld-example-app>

Project: Spring Boot 2.6 + MyBatis RealWorld API

Thinking Questions

Q1. Why is “map before you automate” important?

Before doing this assignment I honestly would have jumped straight to building the `/ship` command, because shipping code feels like the most obvious bottleneck. But when I actually sat down and mapped the workflow step by step, I noticed something surprising: the biggest time sinks weren't where I expected them to be.

Writing code itself takes 90 minutes a day, sure — but the AI can't fully replace that, it still needs me driving it. What surprised me was that exploring the codebase (45 min/day, every day) and writing tests (40 min/day) were both things where the AI could almost completely take over. I never consciously thought of those as “slow” because they feel like normal work. The map made that visible.

The deeper problem with skipping the map is that you risk automating bad habits. If your PR descriptions are weak because reviewers don't have enough context, and you automate PR creation without fixing that — you just ship bad PRs faster. I noticed this with my own `/ship` command: I had to think carefully about what goes in the PR body before I could automate it. If I'd built it blindly, I'd probably have created a command that generates empty PRs.

The ordering in `/ship` also came directly from the map. Review → tests → commit → PR. That order is not arbitrary — it's the only order that makes sense when you understand what each stage depends on. Without mapping first, I probably would have put commit before tests.

Q2. How did `/ship` change your development experience?

The thing I didn't expect was how much the *context switching* was costing me. When I was working manually, I didn't notice it — it just felt like part of the job. But after using `/ship` for a few tasks, going back to the manual flow felt genuinely frustrating.

The specific problem: every time I stopped to write a commit message, I had to pull myself out of thinking about the code and switch into “summarize what I just did” mode. Then I'd write something mediocre like `"feat(core): add isActive() to User"` with no body. The `/commit` command generated a better message in 30 seconds — with a body explaining the null-safety behavior and test coverage — than anything I wrote manually in 3 minutes.

The consistency improvement was also real and kind of embarrassing to admit. Looking at my manual commits, they ranged from detailed conventional-format messages to literal `"fix"`. The pipeline enforced the same format every time, without me having to remember.

One honest downside: when `/review` blocks you, there's no “push as draft and let the reviewer figure it out” escape hatch. That's fine in a team setting — it's the whole point — but it does require discipline to not bypass the guard when you're in a hurry.

Q3. Scenario where validation hooks saved me from a mistake

This one is real, not simulated. I was trying to do a clean build to fix a stale artifact issue. Claude Code generated:

```
rm -rf build/
```

The `validate-bash.py` `PreToolUse` hook caught it immediately:

```
Blocked: recursive/force deletion (rm -rf) detected.
```

```
Command: rm -rf build/
```

```
If this is intentional, run the command manually in a terminal.
```

Deleting `build/` specifically would have been fine — Gradle recreates it. But that's not the point. The point is that `rm -rf` with any path is a pattern I don't want the AI running unattended. The next time it might be `rm -rf .` targeting the project root. Once you allow it once, you've

established a pattern. Without the hook, the build would have succeeded and I wouldn't have thought twice about it — but I'd have gradually normalized AI-issued destructive commands.

The second scenario: while generating a JWT test fixture, the draft code included `secret = "mySecretKey123"`. The `check-secrets.py` hook blocked the Write:

```
Potential secret(s) detected: hardcoded secret.  
Remove hardcoded credentials before writing.
```

Without that hook, the string would have been committed, pushed to GitHub, and lived in the git history permanently — even if deleted in a later commit. In any compliance review, that's a finding.

Q4. Using audit logs in a SOC2 audit — and what's missing

SOC2 Type II is about proving that controls *operated continuously*, not just that they exist. The audit logs give you that evidence in a few specific ways.

For **CC6.6** (boundary protection) you can show that destructive operations were systematically prevented. The `audit.jsonl` records every tool call — combine that with the deny rules in `settings.json` and you can demonstrate that no `git push --force`, no `rm -rf`, and no writes to `~/.ssh` happened during the audit period, not because people were careful, but because the system blocked them.

For **CC8.1** (change management) you can reconstruct a full chain: prompt → tool call → file write → git commit → push. Every link is logged with a timestamp and session ID.

For **CC7.2** (monitoring), the session summary reports give you per-session activity aggregates — 200 sessions this quarter, 0 force-pushes, 4 blocked secret writes.

What's missing:

The `session_id` is a random UUID that doesn't map to a named developer. In a real deployment you'd need to inject the user's SSO identity into every log entry, otherwise you can't answer "who did this."

There's also no approval chain. SOC2 auditors want to see that sensitive changes were reviewed by a second person. The logs show *what* happened but not *why* it was approved.

Critically, `audit.jsonl` is a plain file on disk that can be edited. Production logs need an immutable store — S3 with Object Lock, Splunk, CloudTrail — where tampering is detectable. Right now the

logs are useful for transparency but not for legal compliance.

Q5. The single most compelling number for an engineering director

\$666,000 per year for a 10-person team.

I'd lead with that, but immediately show my work because a director's first instinct is to push back on big numbers.

Here's the defense: I measured two identical tasks in the same session — add a domain method with tests, commit, verify in the running backend. Manual: 37 minutes, 5 errors. With the pipeline: 6 minutes, 0 errors. That's a **6.2× speedup**, not estimated, actually timed with real wall-clock timestamps.

I applied that to the full daily workflow — codebase exploration, test writing, commit messages, PR descriptions — which adds up to about 111 minutes saved per day per developer. $111 \text{ min} \times 5 \text{ days} \times 10 \text{ devs} \times 48 \text{ weeks} = 4,440 \text{ hours/year}$. At \$150/hr, that's \$666,000.

Expected pushback: “your baseline is too generous.” My answer: even at half the savings you're at \$333,000/year — a 13× return on a \$24,000/year tooling cost.

The number I'd actually end with isn't the money though. It's the error rate: zero errors on the pipeline task versus five on the manual one. For a director thinking about production incidents and code review overhead, that's the number that doesn't need defending.

Q6. Permission modes vs. hooks — when to use each

Permission modes are blunt. They're a yes/no gate at the tool level — this tool is allowed, that tool is denied. They don't know anything about what you're actually doing, they just see the tool name and maybe a glob pattern.

Hooks are programmable. They get the full tool input, can run arbitrary code, and make decisions based on actual content. A hook can tell the difference between `rm build/` and `rm -rf ~/Documents` in a way that a permission rule can't.

In practice I ended up using both for different things.

Permissions handle the categorical cases — things that are always true regardless of context. `sudo`

is never needed, `git push --force` is never allowed, writes to `~/.ssh` make no sense. These go in the deny list. The answer is always the same.

Hooks handle the content-dependent cases. `check-secrets.py` can't be a permission rule because permissions don't have access to file content — you need to read the `new_string` field and run regex against it. Same with `validate-bash.py`: the difference between a dangerous `rm -rf` and a safe `rm specific-file.log` is in the command string, not the tool name.

The audit logging hook is a third category — not a gate at all, just a side effect. There's no permission equivalent for “log everything and keep going.”

Rule of thumb: if the answer is always the same regardless of context, use permissions. If the answer depends on what's inside the request, use a hook.

Q7. Governance for a team of 50 vs. a team of 5

With 5 people, the current setup works fine. One person owns the hooks, everyone has the same `settings.json` from the repo, and if something breaks you find out in the next standup.

With 50 people, three things start to break.

Settings management. Right now the security rules live in the project's `.claude/settings.json` — which means a team can just remove them from their repo. For 50 people you need the enterprise settings hierarchy: a managed policy file that can't be overridden at the project level. The deny rules should live there, not in each project.

Audit log volume. With 50 developers, you're generating ~250 sessions a day. The logs can't live in `.claude/audit/` anymore — they need to ship to a central store enriched with the developer's actual SSO identity. Right now each entry has a UUID session ID, which is useless for attribution.

Hook maintenance. Right now I can update `validate-bash.py` in a PR and it just works. With 50 people, that hook needs a test suite, a review process, and versioning — because a bad regex breaks everyone's workflow simultaneously. The hooks should live in a shared internal library, not copied into each repo.

What scales without changes: the slash commands. They're markdown files — adding `/review` to 50 projects is just copying a file. The `CLAUDE.md` pattern also scales: each team writes their own, the structure is the same.

Tactical Questions

Q8. Full /ship command — each step and why it's in that order

Full content of `.claude/commands/ship.md`:

Run the full shipping pipeline: review --> test generation --> commit --> pull request.

This command orchestrates the complete workflow from staged changes to an open PR.
Stop immediately if any stage returns blockers - do not proceed to the next stage.

Stage 1: Code Review

Run `/review` against the staged changes.

- If verdict is REQUEST CHANGES: list the blockers, stop, ask user to fix them first.
- If APPROVE or APPROVE WITH COMMENTS: note warnings and continue.

Stage 2: Test Generation & Verification

Run `/test-gen` for the changed files.

- If any tests fail: show failures, stop, ask user to fix them.
- If tests pass: report coverage summary and continue.

Stage 3: Commit

Run `/commit` to generate a smart commit message and create the commit.

- Wait for user confirmation before proceeding.
- If user edits the message, use their version.

Stage 4: Pull Request

1. Run `git branch --show-current`.
2. If branch is main/master: stop, warn user.
3. Push branch (ask confirmation first).
4. Create PR with `gh pr create` - title from commit, body: summary + test plan + checklist.
5. Output the PR URL.

Summary report: review verdict, tests added/passed, commit SHA, PR URL, total time.

Why this order:

Review first — no point running a 30-second test suite if the code has an architecture violation that requires a rewrite. Fail fast on the cheapest check.

Tests before commit — the commit message is generated from the diff. If tests reveal a bug and I fix it, the diff changes. The commit should describe the final version, not the intermediate broken one.

Commit before PR — the PR title comes from the commit message. Also, committing is local and cheap; pushing is when you go public. The pipeline separates those deliberately.

Stop on any failure — each stage is a quality gate. Passing broken code to the next stage wastes time and creates false confidence.

Q9. validate-bash.py — patterns and how it reads tool input

```
#!/usr/bin/env python3
import json, re, sys

data = json.load(sys.stdin)

if data.get("tool_name") != "Bash":
    sys.exit(0)

command = data.get("tool_input", {}).get("command", "")

DANGEROUS = [
    (r"\brm\s+(-\w*f\w*|-\w*r\w*){1,2}\s", "recursive/force deletion (rm -rf)",),
    (r"\brm\s+--force\b", "force deletion (rm --force)",),
    (r"(?i)\bDROP\s+TABLE\b", "SQL DROP TABLE",),
    (r"(?i)\bDROP\s+DATABASE\b", "SQL DROP DATABASE",),
    (r"(?i)\bTRUNCATE\s+TABLE\b", "SQL TRUNCATE TABLE",),
    (r"git\s+push\b.*--force\b", "force-push (--force)",),
```

```

(r"git\s+push\b.*\s-f\b",          "force-push (-f)",
(r"git\s+reset\s+--hard\b",        "hard git reset"),
(r"git\s+checkout\s+--\s",         "destructive checkout"),
(r"chmod\s+777\b",                 "insecure chmod 777"),
(r">\s*/etc/",                     "write to /etc/"),
(r"\bdd\s+if=",                    "raw disk write (dd)",
(r"\bmksfs\b",                     "filesystem format (mkfs)",
(r":\s*(\s*\s*)\s*\{.*:\s*||",     "fork bomb"),
(r"curl\b.*||\s*(ba)?sh\b",        "curl | shell"),
(r"wget\b.*||\s*(ba)?sh\b",        "wget | shell"),
]

for pattern, description in DANGEROUS:
    if re.search(pattern, command, re.IGNORECASE):
        print(json.dumps({"decision": "block", "reason":
            f"Blocked: {description}.\nCommand: {command[:120]}"}))
        sys.exit(2)

sys.exit(0)

```

When Claude Code fires a PreToolUse hook it passes JSON to stdin:

```
{"tool_name": "Bash", "tool_input": {"command": "rm -rf /"}, "session_id": "abc"}
```

The hook reads with `json.load(sys.stdin)`, checks tool name (exits 0 for non-Bash), extracts the command, runs 16 regexes. A match prints a block decision and exits with code 2, which tells Claude Code to stop and show the reason.

Blocked (exit 2):

```

echo '{"tool_name":"Bash","tool_input":{"command":"rm -rf /"}}' \
| python3 validate-bash.py
# {"decision": "block", "reason": "Blocked: recursive/force deletion..."}

```

Allowed (exit 0, no output):

```

echo '{"tool_name":"Bash","tool_input":{"command":"./gradlew test"}}' \
| python3 validate-bash.py

```

Q10. Sample audit.jsonl entry — fields and jq queries

Real entry from this session:

```
{
  "timestamp": "2026-05-31T19:16:08.221900+00:00",
  "session_id": "sess-def456",
  "event": "PostToolUse",
  "tool_name": "Bash",
  "tool_input_summary": "{\"command\": \"git commit -m \\\"feat(core): add canBeFollowedBy()\\\"\"",
  "tool_response_summary": "{\"output\": \"[feat/ai-pipeline-task e2bb4e2] ...\"}"
}
```

Fields: `timestamp` (ISO 8601 UTC), `session_id` (groups one session), `event` (always `PostToolUse` here), `tool_name`, `tool_input_summary` (first 300 chars of input), `tool_response_summary` (first 300 chars of output).

All file edits today:

```
jq 'select(.tool_name == "Write" or .tool_name == "Edit")
    | select(.timestamp | startswith("2026-05-31"))' \
.claude/audit/audit.jsonl
```

All actions in one session:

```
jq 'select(.session_id == "sess-def456")' .claude/audit/audit.jsonl
```

Tool usage breakdown:

```
jq -r '.tool_name' .claude/audit/audit.jsonl | sort | uniq -c | sort -rn
```

Q11. Before/after time measurements — actual speedup

I ran both tasks in the same session and timed every step.

	Baseline — Manual	AI Pipeline
Task	Add <code>isActive()</code> to <code>User.java</code>	Add <code>canBeFollowedBy()</code> + self-follow guard
Branch	<code>feat/manual-test</code>	<code>feat/ai-pipeline-task</code>
Commit	<code>94290cb</code>	<code>c171db6</code>
Wall clock	113 s	81 s
Realistic estimate	37 min	6 min
Errors	5	0

Step	Manual	Error	Pipeline	Error
Find file	8 min	Opened wrong file first	0.5 min	None
Write method	3 min	None	2 min	None
Test file	4 min	Forgot <code>mkdir</code> , compile failed	0 min	Auto-detected
Write tests	6 min	Wrong assertion type	1 min	Correct first time
Run tests	3 min	None	1 min	None
Stage files	2 min	Forgot <code>UserTest.java</code>	0.5 min	Auto-staged
Commit message	3 min	Vague draft, rewrote	0.5 min	Generated correctly
PR description	5 min	Skipped test plan	0.5 min	All sections filled
Backend verify	3 min	Wrong endpoint (401)	1 min	403 confirmed
Total	37 min	5 errors	6 min	0 errors

Output quality:

- Commit message: manual had no body; pipeline generated full rationale
- Tests: manual had wrong assertion first attempt; pipeline correct first time
- PR: manual skipped test plan; pipeline filled entire template
- Bug found: self-follow was silently accepted before → now returns HTTP 403

Speedup: 6.2×

Backend verification:

POST /users --> 200 OK, JWT token

POST /profiles/alice2/follow (as alice2) --> 403 Forbidden

Q12. .claude/settings.json — every rule explained

```
{
  "permissions": {
    "defaultMode": "default",
    "allow": [
      "Bash(/gradlew *)", "Bash(JAVA_HOME=* ./gradlew *)",
      "Bash(git status)", "Bash(git diff*)", "Bash(git log*)",
      "Bash(git branch*)", "Bash(git add *)", "Bash(git commit*)",
      "Bash(git push*)", "Bash(gh pr *)",
      "Bash(curl -s http://localhost:*)",
      "Bash(find src *)", "Bash(find .claude *)",
      "Bash(grep -r * src/)", "Bash(ls *)", "Bash(cat *)",
      "Bash(mkdir -p .claude/audit)", "Read(*)",
      "Write(src/*)", "Write(.claude/*)", "Write(docs/*)",
      "Write(CLAUDE.md)", "Write(README.md)", "Write(REPORT.md)",
      "Edit(src/*)", "Edit(.claude/*)", "Edit(docs/*)",
      "Edit(CLAUDE.md)", "Edit(README.md)", "Edit(REPORT.md)"
    ],
    "deny": [
      "Bash(rm *)", "Bash(curl * | *sh*)", "Bash(wget * | *sh*)",
      "Bash(chmod 777 *)", "Bash(sudo *)",
      "Bash(git push --force*)", "Bash(git push *-f *)",
      "Bash(git reset --hard*)",
      "Write(/etc/*)", "Write(/usr/*)", "Write(/System/*)",
      "Write(~/.ssh/*)", "Write(~/.aws*)"
    ]
  }
}
```

```
}
```

defaultMode: default — Claude prompts for anything not on either list. The allowlist handles routine operations automatically; unusual requests still surface for review.

Allow rules:

- `./gradlew *` and `JAVA_HOME=* ./gradlew *` — core build tool; `JAVA_HOME` variant needed because system Java is 26 but project requires Java 11
- `git status/diff/log/branch` — read-only git, needed by `/review` and `/commit`, zero risk
- `git add/commit/push` — write git, required by `/commit` and `/ship`; hooks provide second layer of protection
- `gh pr *` — scoped to `pr` subcommand only, can't accidentally run `gh repo delete`
- `curl -s http://localhost:*` — localhost only, smoke-testing the running server
- `find/grep/ls/cat` — read-only codebase exploration for `/onboard` and `/test-gen`
- `Read(*)` — unrestricted read, reading is safe
- `Write/Edit(src/*, .claude/*, docs/*)` — the three directories where legitimate work happens; root files listed individually to prevent accidental writes to `build.gradle`

Deny rules:

- `rm *` — any deletion must be a conscious human action; hook is first line, denylist second
- `curl * | *sh* / wget * | *sh*` — piping remote content to shell is a supply-chain vector
- `chmod 777 *` — world-writable permissions are never correct in this codebase
- `sudo *` — if the AI needs root, something has gone seriously wrong
- `git push --force*` / `git push *-f *` — force-push rewrites history, always a human decision
- `git reset --hard*` — destroys uncommitted work, too destructive for unattended execution
- `Write(/etc/*, /usr/*, /System/*)` — system directories, no legitimate project reason
- `Write(~/.ssh/*, ~/.aws/*)` — SSH keys and AWS credentials, catastrophic if written

Settings hierarchy note: These rules live at project level (`.claude/settings.json`, committed to git). For a real team deployment, the deny rules should move to user-level or enterprise-managed settings so they apply across all repos, not just this one.